Desarrollo de la tienda web NexusPlay con jQuery y PostgreSQL

Carlos Ademar Suarez Quispe

Universidad Privada Domingo Savio

Diseño Web 2

Docente Albaro Moscoso

10 de noviembre de 2026

Contenido

Resumen e introducción 3

Objetivos y alcance 4

Marco teórico 5

Arquitectura y base de datos 6

Catálogo y carrito de compras 7

Administración de productos y categorías 8

Manipulación del DOM con jQuery 9

Formas de pago y confirmación visual 10

Aplicación Android y Google Sheets 11

Despliegue y validación 12

Conclusiones y recomendaciones 13

Referencias 14

Resumen e introducción

Resumen

Se desarrolló NexusPlay, una tienda web académica para explorar videojuegos, administrar productos y actualizar un carrito de compras mediante eventos de teclado y mouse. La interfaz utiliza HTML, CSS, JavaScript y jQuery; el servidor emplea Node.js y Express, y los registros se conservan en PostgreSQL alojado en Neon. El proyecto incorpora filtros por categoría, control de cantidades, administración de imágenes, pagos en efectivo y QR estático, y un QR con monto destinado únicamente a demostración. También integra una aplicación Android que captura avisos de Yape Bolivia y los envía al servidor para su posible asociación con un pedido. Google Sheets conserva una copia de los avisos y de las ventas. Las pruebas verificaron funciones del catálogo, permisos de administración, integridad de datos y conciliación con registros temporales. La confirmación automática depende de recibir un aviso válido y de identificar una coincidencia única; no constituye una consulta bancaria independiente.

Introducción

El laboratorio propone manipular el árbol DOM y aplicar eventos y efectos visuales con jQuery para mantener un carrito actualizado sin recargar toda la página. La necesidad funcional elegida fue organizar la venta de videojuegos y mostrar al comprador una respuesta clara sobre su compra. Para resolverla se conectaron la interfaz, una API y una base de datos persistente. Este informe describe la organización del sistema, sus operaciones principales, la validación realizada y los límites de la integración de pagos.

Objetivos y alcance

Objetivo general

Desarrollar una tienda web para NexusPlay que permita administrar productos, manipular dinámicamente un carrito de compras mediante jQuery y registrar ventas en PostgreSQL, con consulta del estado del pago desde la página principal.

Objetivos específicos

1. Aplicar selectores, eventos delegados y efectos de jQuery para actualizar el catálogo y el carrito.
2. Implementar búsqueda, filtros por categoría, ordenación y controles para aumentar o disminuir cantidades.
3. Registrar y editar productos con imagen, precio, descripción y cantidad, y administrar sus categorías.
4. Conservar ventas, importes y avisos de pago en PostgreSQL mediante una API protegida.
5. Integrar el QR estático, el pago mixto y un QR con monto de demostración claramente identificado.
6. Consultar automáticamente el estado del pedido y copiar avisos y ventas a Google Sheets.

Alcance

Se implementó un prototipo académico desplegado en Render, con catálogo público y panel de administración. Se conservaron los tres productos existentes y se agregaron veinte ejemplos, con precios y cantidades demostrativos. El proyecto no incorpora una API bancaria, emisión de licencias de videojuegos ni liquidación financiera. La recepción de avisos del teléfono se utiliza como evidencia de aplicación para la conciliación, con revisión cuando los datos no permiten identificar un pedido único.

Marco teórico

DOM y programación por eventos

El DOM representa un documento web como un árbol de objetos que puede modificarse desde JavaScript. Su estructura permite acceder a elementos y cambiar contenido, atributos o estilos sin reconstruir manualmente toda la página (MDN Web Docs, s. f.). En NexusPlay, esta capacidad se utiliza para crear fichas de productos, representar las líneas del carrito y mostrar mensajes relacionados con la compra.

jQuery y comunicación asíncrona

jQuery facilita la selección de elementos y la asociación de manejadores de eventos. Las solicitudes asíncronas realizadas con jQuery.ajax permiten intercambiar datos con el servidor y tratar respuestas o errores desde la interfaz (OpenJS Foundation, s. f.-b). El proyecto combina estas solicitudes con eventos input, change, click y submit, además de efectos como fadeIn y slideUp para mostrar cambios al usuario.

API y persistencia relacional

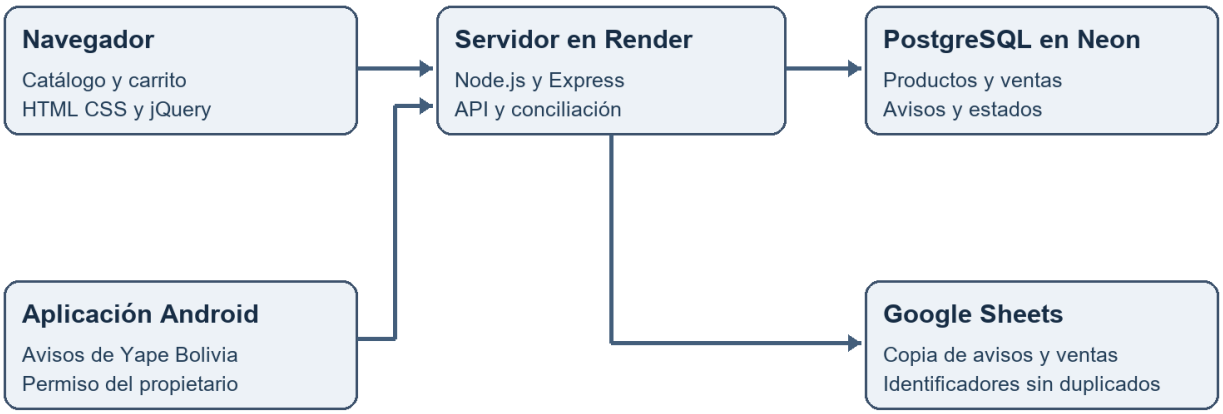
Express define rutas que responden a un método HTTP y una dirección concreta, como GET para consultar o PUT para actualizar recursos (OpenJS Foundation, s. f.-a). PostgreSQL permite agrupar operaciones dentro de una transacción: se confirman en conjunto o se revierten si ocurre un error (PostgreSQL Global Development Group, s. f.). Esta propiedad resulta útil al registrar una venta y descontar sus cantidades de inventario de forma consistente.

Arquitectura y base de datos

La interfaz consulta una API alojada en Render. El servidor valida los datos y trabaja con PostgreSQL en Neon; el navegador no recibe las credenciales de la base. La aplicación Android envía los avisos a una ruta específica y Google Sheets recibe una copia de los registros para su consulta. La Figura 1 resume estas relaciones.

Figura 1

Componentes y comunicación del sistema



Nota. Elaboración propia. El teléfono aporta avisos de notificación; no existe una conexión directa con una API bancaria.

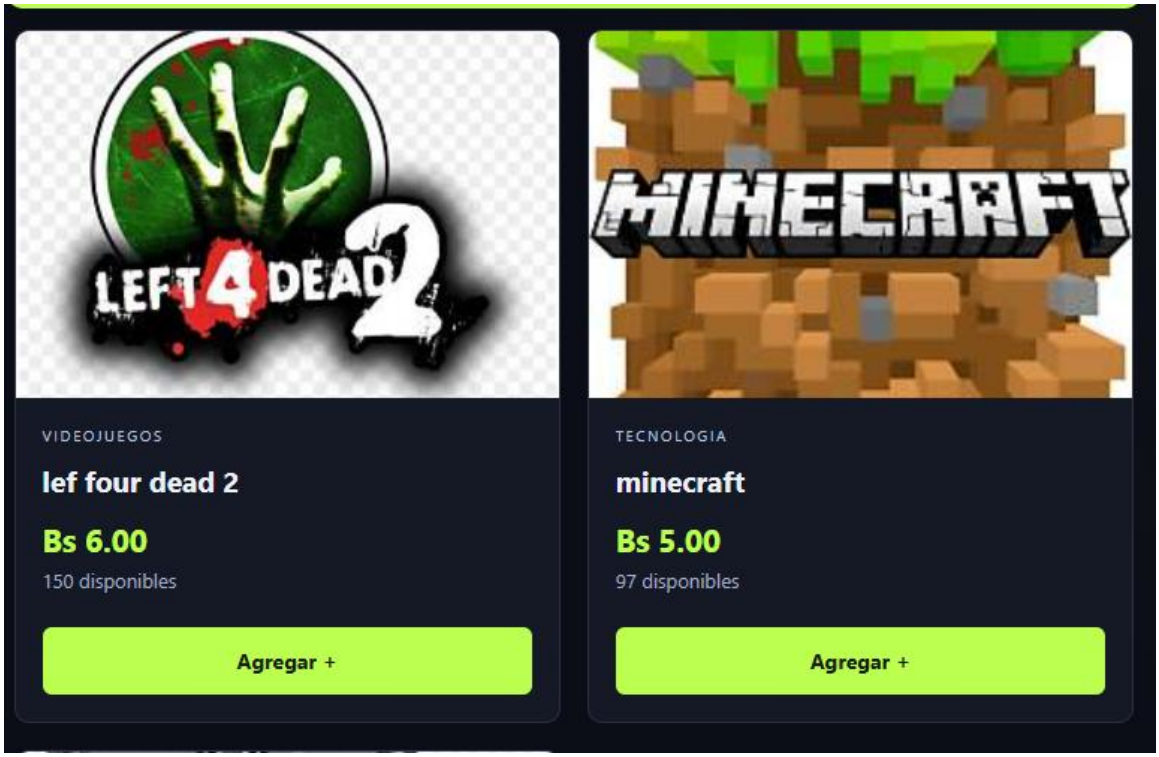
El esquema tienda relaciona categorias y productos mediante una clave foránea. ventas conserva el detalle vendido y los importes; notificaciones_yape almacena cada aviso, su fecha y el pedido asociado. configuracion mantiene la imagen QR estática. Las ventas incluyen una clave aleatoria para consultar su estado sin publicar el historial completo de los demás compradores.

Catálogo y carrito de compras

El catálogo ampliado contiene 23 productos con imagen, categoría, precio y cantidad. La búsqueda se combina con el filtro por categoría y la ordenación por nombre o precio. Un contador muestra los resultados y Limpiar filtros restaura la vista general.

Figura 2

Presentación del catálogo durante el desarrollo



Nota. Captura proporcionada por el autor, anterior a la ampliación del catálogo y al filtro por categorías.

El carrito muestra miniatura, precio unitario, cantidad y subtotal. Los botones de suma y resta respetan el inventario disponible y actualizan el total. El comprador puede quitar productos o vaciar el carrito después de confirmar la acción.

Administración de productos y categorías

El administrador puede crear categorías y registrar productos con nombre, descripción, precio, cantidad e imagen. El botón Editar carga los datos existentes y permite conservar la fotografía o sustituirla. Guardar cambios actualiza el producto identificado por su clave; Cancelar edición regresa al formulario de alta. La modificación requiere credenciales de administración en producción.

Figura 3

Ejemplo resumido de actualización de un producto

Actualización protegida del producto

```
app.put('/api/productos/:id', async (req, res) => {
  // Validar los campos y la categoría registrada.
  await pool.query(
    'UPDATE tienda.productos SET nombre=$1, precio=$2' +
    ' WHERE id=$3',
    [nombre, precio, req.params.id]
  );
  res.json({ actualizado: true });
});
```

Nota. Adaptación didáctica del código del proyecto. La implementación completa también valida campos, comprueba la categoría y administra la imagen mediante parámetros SQL.

La eliminación de categorías requiere una confirmación. PostgreSQL impide borrar una categoría que tenga productos asociados, por lo que antes deben reasignarse. General se conserva como categoría predeterminada. Las consultas de modificación utilizan parámetros; de esta manera los valores escritos en el formulario no se concatenan dentro de la sentencia SQL.

Manipulación del DOM con jQuery

Los eventos se organizan según la interacción del usuario. input actualiza la búsqueda; change aplica categoría y ordenación; click añade, resta o quita unidades; submit registra formularios. Como varias fichas y botones se crean después de cargar la página, los eventos del catálogo y del carrito se delegan en sus contenedores. Así los elementos nuevos mantienen el mismo comportamiento.

Figura 4

Selección y filtrado dinámico de productos

Eventos y actualización del catálogo

```
$('#filtrar-categoria,#orden-productos')
  .on('change', catalogo);
const visibles = productos.filter(p =>
  (!categoria || p.categoria === categoria) &&
  normalizarBusqueda(p.nombre).includes(texto)
);
$('#catalogo').empty();
visibles.forEach(p => mostrarProducto(p));
```

Nota. Fragmento adaptado para explicar el mecanismo. mostrarProducto representa la construcción de la ficha; en el código final se emplea el recorrido de los resultados filtrados.

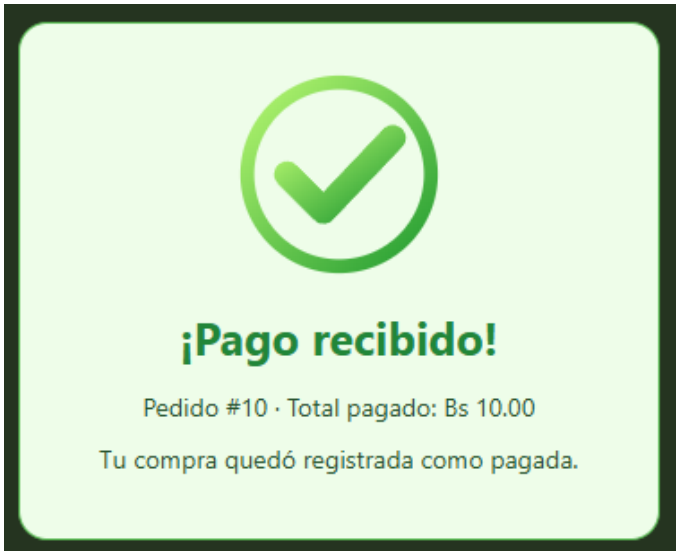
El catálogo se vacía y se vuelve a representar con los productos que cumplen los filtros. El carrito sigue un proceso semejante: modifica su arreglo y dibuja nuevamente cantidades y subtotales. Los efectos visuales se usan como apoyo a la interacción. Las validaciones del navegador mejoran la experiencia, pero el servidor vuelve a verificar importes, cantidades y disponibilidad antes de guardar una venta.

Formas de pago y confirmación visual

Se mantienen cuatro opciones: efectivo, QR estático, efectivo más QR y QR con monto de demostración. El QR demostrativo contiene un texto con el importe del carrito; no realiza cobros, crea ventas ni descuenta inventario. El QR estático conserva la imagen bancaria configurada por el administrador.

Figura 5

Comprobante visual del estado pagado



Nota. Captura de la interfaz del proyecto. El check representa el estado del servidor y no una validación bancaria independiente.

La página consulta el pedido cada ocho segundos y muestra el check cuando el servidor responde pagado. En la conciliación por aviso se exige nombre completo, importe exacto, fecha compatible y un único pedido candidato. En pagos mixtos, el aviso puede reconocer el QR, pero el efectivo requiere comprobación administrativa.

Aplicación Android y Google Sheets

NotificationListenerService permite recibir eventos de notificaciones cuando el propietario habilita el acceso correspondiente (Google, s. f.-a). La aplicación del proyecto filtra el paquete de Yape Bolivia antes de leer el contenido. Guarda los avisos en el teléfono, intenta enviarlos por HTTPS y conserva los pendientes para reintentar cuando una conexión no se completa.

Figura 6

Secuencia de captura de un aviso en el teléfono

Captura y envío desde Android

```
override fun onNotificationPosted(sbn: StatusBarNotification) {
  if (sbn.packageName != paqueteConfigurado) return
  val aviso = crearAviso(sbn)
  guardarEnTelefono(aviso)
  enviarEnseguida()
}
```

Nota. Representación simplificada del flujo implementado. El código utiliza un identificador para evitar duplicados y no confirma una venta únicamente por recibir texto.

Google Apps Script permite publicar una aplicación web que atienda solicitudes, por ejemplo mediante doPost (Google, s. f.-b). En NexusPlay, el servidor envía lotes a esta aplicación y Google Sheets identifica cada fila por su registro. Se distinguen los avisos de Yape de las filas NexusPlay venta. Copiar una venta pagada refleja el estado del sistema; no transforma ese registro en una notificación bancaria.

Despliegue y validación

El código se conserva en GitHub y se despliega en Render con npm start y variables de entorno. Render admite los comandos de instalación y arranque propios de la aplicación (Render, s. f.). La ruta de salud permitió comprobar la versión activa y su conexión con la base.

Tabla 1

Comprobaciones efectuadas durante el desarrollo

Función	Resultado observado
Catálogo y filtros	Carga de 23 productos y combinación de categoría, búsqueda y orden.
Carrito	Cambio de cantidad y recálculo de subtotales sin compra real.
Edición de productos	Actualización protegida y conservación o sustitución de imagen.
Categorías	Bloqueo de borrado con productos y protección de General.
QR demostrativo	Importe calculado por el servidor sin venta ni descuento de stock.
Conciliación	Pruebas temporales de monto, nombre, fechas, duplicados y ambigüedad.
Google Sheets	Copia de ventas y comprobación de que un reenvío no las duplica.

La conciliación se probó con registros temporales revertidos. En un teléfono real, la captura y transmisión dependen del permiso, la conexión y el servicio de Android. Una compra visible en administración no equivale a recibir su aviso de Yape; si no existe una coincidencia válida, permanece pendiente.

Conclusiones y recomendaciones

Conclusiones

El proyecto permitió aplicar manipulación del DOM, programación por eventos y comunicación asíncrona en una necesidad concreta: mantener un carrito de compras actualizado y conservar sus operaciones en una base relacional. La separación entre catálogo, administración, API y persistencia facilita comprender qué componente procesa cada acción.

La administración de categorías y productos, junto con los filtros y los controles de cantidad, amplió el ejercicio inicial sin depender de una recarga completa por cada interacción. Las pruebas mostraron la importancia de validar tanto en la interfaz como en el servidor, especialmente al modificar imágenes, cantidades e importes.

La integración de pagos requiere distinguir tres hechos: registrar un pedido, recibir un aviso del teléfono y reconocer una coincidencia con la venta. Mostrar un check solo cuando el estado del servidor es pagado evita presentar como confirmada una operación pendiente. Google Sheets funciona como copia de consulta y no como autoridad bancaria.

Recomendaciones

Se recomienda mantener las claves fuera del código y de las capturas, registrar el motivo de las asociaciones de pago y comprobar los errores del lector sin repetir transferencias innecesarias. Para una tienda de producción conviene incorporar autenticación por usuarios, auditoría de cambios, copias de seguridad y un proveedor de pagos con confirmaciones verificables. Los productos, precios y cantidades de ejemplo deben revisarse antes de emplearse en operaciones comerciales.

Referencias

Google. (s. f.-a). *NotificationListenerService.* Android Developers. https://developer.android.com/reference/android/service/notification/NotificationListenerService

Google. (s. f.-b). *Web apps.* Google for Developers. https://developers.google.com/apps-script/guides/web

MDN Web Docs. (s. f.). *Document Object Model (DOM).* https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model

OpenJS Foundation. (s. f.-a). *Basic routing.* Express. https://expressjs.com/en/starter/basic-routing/

OpenJS Foundation. (s. f.-b). *jQuery.ajax().* jQuery API Documentation. https://api.jquery.com/jQuery.ajax/

PostgreSQL Global Development Group. (s. f.). *Transactions.* PostgreSQL Documentation. https://www.postgresql.org/docs/current/tutorial-transactions.html

Render. (s. f.). *Deploy a Node Express app on Render.* https://render.com/docs/deploy-node-express-app